

Project Documentation: VeriRISC Single-Bidirectional-Port Memory

Digital Design Lab

Project Overview

This lab implements a **Single-Bidirectional-Port Memory Unit** for the VeriRISC CPU in Verilog HDL [?]. The CPU uses a shared memory block for both instructions and data, employing a single 8-bit bidirectional data bus (**data**) and separate write (**wr**) and read (**rd**) control signals [?]. Synchronous write operations occur on the positive clock edge (**posedge clk**), while read operations operate asynchronously using continuous assignments driven by the read enable signal [?, ?].

Specifications & Functionality

- **Address Width (AWIDTH):** Parameterized to 5 bits ($2^5 = 32$ memory locations) [?, ?].
- **Data Width (DWIDTH):** Parameterized to 8 bits [?, ?].
- **Synchronous Write:** Memory content updates at `memory[addr]` on `posedge clk` when `wr = 1` [?, ?].
- **Combinational Read:** Continuous assignment drives `data = memory[addr]` when `rd = 1`, driving high-impedance (`8'bz`) when `rd = 0` [?, ?].

Port Summary

Port Name	Direction	Type	Width	Description
clk	Input	wire	1	Clock input signal [?, ?]
wr	Input	wire	1	Write enable signal [?, ?]
rd	Input	wire	1	Read enable signal [?, ?]
addr	Input	wire	AWIDTH (5)	Memory location address input [?, ?]
data	Inout	wire	DWIDTH (8)	Bidirectional data line [?, ?]

Table 1: Memory Module Port Interfaces

Verilog Source Code

Design Module (`memory.v`)

```
module memory #(
    parameter AWIDTH = 5,
    parameter DWIDTH = 8
) (
```

```

input  wire          clk,
input  wire          wr,
input  wire          rd,
input  wire [AWIDTH-1:0] addr,
inout  wire [DWIDTH-1:0] data
);

// Register array defining depth 2^AWIDTH and width DWIDTH
reg [DWIDTH-1:0] mem [0:(1<<AWIDTH)-1];

// Synchronous Write Operation
always @(posedge clk) begin
    if (wr) begin
        mem[addr] <= data;
    end
end

// Continuous Read Operation with High-Impedance tristate output
assign data = rd ? mem[addr] : {DWIDTH{1'bz}};

endmodule

```

Testbench Module (memory_test.v)

```

`timescale 1ns / 1ps

module memory_test;

    localparam integer AWIDTH = 5;
    localparam integer DWIDTH = 8;

    reg          clk;
    reg          wr;
    reg          rd;
    reg [AWIDTH-1:0] addr;
    wire [DWIDTH-1:0] data;
    reg [DWIDTH-1:0] rdata;

    assign data = rdata;

    memory #(
        .AWIDTH ( AWIDTH ),
        .DWIDTH ( DWIDTH )
    ) memory_inst (
        .clk ( clk ),
        .wr  ( wr  ),
        .rd  ( rd  ),
        .addr ( addr ),
        .data ( data )
    );

    task expect;
        input [DWIDTH-1:0] exp_data;
        if (data !== exp_data) begin
            $display("TEST FAILED");
            $display("At time %0d addr=%b data=%b", $time, addr, data);
        end
    endtask

```

```

        $display("data should be %b", exp_data);
        $finish;
    end else begin
        $display("At time %0d addr=%b data=%b", $time, addr, data);
    end
endtask

initial repeat (67) begin #5 clk=1; #5 clk=0; end

initial @(negedge clk) begin : TEST
    reg [AWIDTH-1:0] addr;
    reg [DWIDTH-1:0] data;

    addr=0; data=-1;
    $display("Writing addr=%b data=%b", addr, data);
    wr=1; rd=0; memory_test.addr=addr; rdata=data; @(negedge clk);

    addr=-1; data=0;
    $display("Writing addr=%b data=%b", addr, data);
    wr=1; rd=0; memory_test.addr=addr; rdata=data; @(negedge clk);

    addr=0; data=-1;
    $display("Reading addr=%b data=%b", addr, data);
    wr=0; rd=1; memory_test.addr=addr; rdata='bz; @(negedge clk) expect
        (data);

    addr=-1; data=0;
    $display("Reading addr=%b data=%b", addr, data);
    wr=0; rd=1; memory_test.addr=addr; rdata='bz; @(negedge clk) expect
        (data);

    $display("Writing ascending data to descending addresses");
    addr=-1; data=0;
    while (addr) begin
        wr=1; rd=0; memory_test.addr=addr; rdata=data; @(negedge clk);
        addr=addr-1;
        data=data+1;
    end

    $display("Reading ascending data from descending addresses");
    addr=-1; data=0;
    while (addr) begin
        wr=0; rd=1; memory_test.addr=addr; rdata='bz; @(negedge clk)
            expect(data);
        addr=addr-1;
        data=data+1;
    end

    $display("TEST PASSED");
    $finish;
end

endmodule

```

Simulation & Verification

Execution Command

Run the design and testbench through Cadence Xcelium [?]:

```
xrun memory.v memory_test.v
```

Expected Simulation Output

The simulation verifies address writing and reading across the memory array [?]:

```
Writing addr=00000 data=11111111
Writing addr=11111 data=00000000
Reading addr=00000 data=11111111
At time 350 addr=00000 data=11111111
Reading addr=11111 data=00000000
At time 360 addr=11111 data=00000000
Writing ascending data to descending addresses
Reading ascending data from descending addresses
At time 370 addr=11111 data=00000000
At time 380 addr=11110 data=00000001
At time 390 addr=11101 data=00000010
At time 400 addr=11100 data=00000011
At time 410 addr=11011 data=00000100
At time 420 addr=11010 data=00000101
At time 430 addr=11001 data=00000110
At time 440 addr=11000 data=00000111
At time 450 addr=10111 data=00001000
At time 460 addr=10110 data=00001001
At time 470 addr=10101 data=00001010
At time 480 addr=10100 data=00001011
At time 490 addr=10011 data=00001100
At time 500 addr=10010 data=00001101
At time 510 addr=10001 data=00001110
At time 520 addr=10000 data=00001111
At time 530 addr=01111 data=00010000
At time 540 addr=01110 data=00010001
At time 550 addr=01101 data=00010010
At time 560 addr=01100 data=00010011
At time 570 addr=01011 data=00010100
At time 580 addr=01010 data=00010101
At time 590 addr=01001 data=00010110
At time 600 addr=01000 data=00010111
At time 610 addr=00111 data=00011000
At time 620 addr=00110 data=00011001
At time 630 addr=00101 data=00011010
At time 640 addr=00100 data=00011011
At time 650 addr=00011 data=00011100
At time 660 addr=00010 data=00011101
At time 670 addr=00001 data=00011110
TEST PASSED
```